

RepCRec Project Report

Overview

This simulator implements the RepCRec assignment from `handDB2-4-11.pdf` and follows the design in `Proposal.md`. It models a centralized Transaction Manager (TM) coordinating ten data sites with replicated and non-replicated variables under Serializable Snapshot Isolation (SSI) and the Available Copies (ROWAA) algorithm.

Requirements Trace

****Data topology:**** Variables $x_1..x_{20}$, even indexes on all sites, odd indexes on a single site $1 + (i \bmod 10)$, initial value $10 * i$.

****Execution model:**** Each input line advances the logical clock. TM never fails and parses commands from file or stdin.

****Reads:**** Non-replicated variables read from their unique site if up; otherwise the transaction waits. Replicated variables read the freshest committed version before transaction start from any site that has been continuously up since that commit; if every replica was down in that window the transaction aborts; otherwise it waits for a readable copy.

****Writes:**** Buffered in the transaction workspace. On commit, writes propagate to all currently up sites holding the variable (ROWAA). Down sites remain stale until later recovery plus a fresh commit.

****Commit validation (SSI):**** First-committer-wins on WW conflicts, and dangerous structure detection via consecutive RW antidependencies; any site that a transaction wrote to and that failed before commit forces an abort.

****Failure/recovery:**** Fail clears MVCC history (copies become unreadable until refreshed). Recovery marks non-replicated copies readable immediately and replicated copies stale until updated. Waiting transactions re-attempt deferred operations when sites recover.

****Output:**** Reads print `x#: value`; commits/aborts print the transaction outcome; writes log the sites updated; `dump` prints per-site committed snapshots.

Implementation Notes

****Modules:**** `transaction_manager.py` (orchestration, SSI checks, failure history, deferred waits), `transaction.py` (state, deferred op queue), `data_site.py` (site storage and stale flags), `variable_copy.py` (MVCC history with reset on failure), `parser.py` (command normalization), `main.py` (CLI).

****SSI tracking:**** RW antidependency edges are recorded on commit; a transaction with an incoming RW edge from a transaction that itself has outgoing RW edges triggers the dangerous-structure abort.

****Available copies:**** Writes record touched sites for later failure aborts; commit spreads writes only to up sites, logging affected sites.

****Wait handling:**** Reads/writes issued while waiting are queued and replayed after recovery to respect input order.

Testing

Run `./run_all_tests.sh` to execute all provided scenarios:

`test_basic`: baseline read/write.

``test_ww_conflict``: first-committer-wins.
``test_rw_conflict``: SSI RW detection.
``test_snapshot``: snapshot visibility.
``test_site_failure``: fail/recover with stale replicas.
``test_replicated``: available  copies reads/writes.
``test_readonly``: read  only snapshot behavior.
``test_comprehensive``: mixed concurrency, failures, recovery.

All tests pass locally on the ``dev`` branch.

How to Run

`python main.py`

`python main.py # interactive mode`

Requires Python 3.9+ standard library only.

Alignment with Proposal

Architecture (Parser -> TM -> Sites -> VariableCopy) matches the block diagram in ``Proposal.md``.

ROWAA replication, MVCC snapshots, failure history, and stale  flag recovery adhere to the described plan.

Outputs and waiting semantics satisfy the grading PDF requirements (value logging, commit/abort reasons, dump format).